

实验报告 / Experiment Report — Harness Evolution

机制验证路线

日期 / Date: 2026-08-18
覆盖范围 / Scope: 2026-08-11 至 2026-08-18
Benchmarks: QFBench、QuantCodeEval
报告边界 / Boundary: engineering mechanism validation, 不是 benchmark-wide 或论文级最终结果

结论先行 / Executive Summary

我们现在整个进度可以概括为：已经在 QuantCodeEval T26 这一个 task 上，跑通了 Evolver 利用 rich evidence 找组件、修改完整 harness、再让 fresh Worker 得到 17/17 的路径；但还没有验证 Evolver 能否从 shell-only H0 全自主开始，也没有实现面向全量 task 的异步 scheduler。

更具体一点，目前已经发生的是：Evolver 能看到过去 Worker 做错了什么，也能看到自己上一轮改了什么；它不再只能改 prompt，而是能新增 tool、middleware、skill 和 agent binding；这些组件在真实 Worker 里确实被调用过；在 T26 上，answer-rich feedback 帮助它把结果稳定推到两次 16/17，随后一个 Evolver-produced harness 又在 fresh Worker 中达到一次 17/17、reward 1。

但这几件事还没有连成一次完全自主的 run。当前 17/17 之前的 AP-1 repair probe、candidate promotion 和 final evaluation 是实验者编排的。所以下一步不是继续人工加 failure class，而是让 Evolver 自己决定要跑什么 probe、看完结果以后再决定怎么改。这就是 AP-2M。AP-2M 成立后，才进入从 H0 开始的 AP-3。

1. 实验过程与结果 / Process & Results

1.1 整条机制路线 / Linear Mechanism Route

下面这张图从上到下表示我们的实际推进顺序。左边写“项目走到了哪一步”，右边写“这一阶段到底验证了什么”。

整体进度（从上到下）	这一阶段验证的机制	实际结果
① 先让 Evolver 能正常结束一次搜索 ↓	Probe、checkpoint、 ACT/ABSTAIN 终局协议	QFBench ME7、ME10 都能在证据不足时合法 ABSTAIN；不再因为控制流程坏掉而被迫乱改 harness。
② 再让 Evolver 不只修改 prompt ↓	Full-harness mutation： tool、skill、middleware、 agent config、system prompt	QuantCodeEval v2 已经多次产生 executable multi-file candidate；component smoke、admission 和 Worker activation 都跑通过。

整体进度（从上到下）	这一阶段验证的机制	实际结果
③ 让下一轮看到上一轮做过什么 ↓	Runtime history: 保留 candidate、diff、Worker artifact、trace、score 和失败结果	T18 下一轮能用上一轮无收益结果排除 warmup hypothesis; T24 能用不可达 hook 的结果改成可达 middleware。
④ 让历史帮助 Evolver 选组件, 而不是只记日志 ↓	Quant-specific component search 与 REUSE / REFINE / COMPOSE / ABSTAIN	T12 的历史揭示 free-form probe 会 self-confirm; public-quantity binding 后两次 16/16, T19 18/18。最终 binding 当时仍由实验者 seed。
⑤ 把同一套搜索接口放到两个 benchmark 上 ↓	Cross-benchmark experience adapter	QFBench evidence 不够时选择 ABSTAIN ; QuantCodeEval T26 evidence 较强时生成五文件 candidate。接口可用, 但 T26 两次为 14/17、12/17, 收益不稳定。
⑥ 给 Evolver rich evidence, Worker 继续 blind ↓	Optimize-task answer-rich attribution; answer-blind Worker	T26 Evolver 找到 quant-contract auditor; 两个独立 Worker 都达到 16/17, B5/B9 按预测转为 PASS。
⑦ 验证 component 是否真的能带来二元收益 ↓	Paired repair、fresh Worker、official verifier	AP-1 中 parent 为 12/17、candidate 为 14/17; 同一 candidate harness 的 fresh Worker 随后达到 17/17、reward 1。
⑧ 接下来让 Evolver 自己选择实验 ↓	AP-2M: self-selected probe + feedback-driven second decision	尚未运行。这是当前最直接的机制缺口。
⑨ 再从 shell-only H0 自主开始 ↓	AP-3: H0 evidence → component → probe → feedback → final candidate	尚未运行。只有这一步成立, 才能说系统具备初步 H0 bootstrap 能力。
⑩ 最后才扩展到多 task 和 scheduler	Repeat、第二个 task、QFBench、异步成本调度、完整 test set	尚未实现。当前先不让 scaling 工程干扰单 task 机制验证。

1.2 这条路线具体是怎么走过来的

第一步, 我们先解决 Evolver 会不会停不下来、或者明明没有证据还硬改的问题。8 月 11 日的 QFBench ME1–ME10 主要不是为了涨分, 而是为了让真实 model run 能完成 probe、保存 checkpoint, 最后给出可信的 ACT 或 ABSTAIN。ME7 和 ME10 最终都给出了合法 ABSTAIN。这说明 control loop 已经能工作; 它当时没有产生 candidate, 所以不能说 harness 有提升。ME1–ME10 共记录 172 logical requests、至少 5,798,107 tokens 和 USD 0.3520366696。

第二步，我们把搜索对象从 prompt 扩大成整个 Worker harness。 8 月 12 日最早的五轮 QuantCodeEval canary 仍然只改 `systemprompt.md`。它没有提高 binary reward，但第 5 轮把 T24 从 59 requests 后没有 artifact，变成 10 个 T24 requests 内交付 artifact 并得到 15/17。这说明 prompt mutation 可以缓解 completion 问题，但不足以修 quant semantics。随后 v2 允许 Evolver 新增 executable tool、tool description、agent registration、skill 和 middleware，也把每轮 candidate 和结果留给下一轮。r8 的四文件 quant audit component 在真实 Worker 中被调用，T24 从 H0 15/17 到 16/17，T16 仍为 18/18。r9 继续堆 static rules 后反而让 T16 变成 3/18、T24 没有 artifact，因此我们停止了“继续加静态规则”的路线。

第三步，我们开始保存 Worker 的运行经验，而不只是保存最终分数。 8 月 13 日，Evolver 能看到 Worker 读了什么、什么时候改 strategy、跑了什么 public probe，以及 probe 结果。四个 T12 历史分支最后都通过了 Worker 自己写的 probe，但 official score 从 8/16 到 16/16 不等。这个对比很关键：问题不是 Worker 完全没测试，而是它自己决定解释、再自己写 expected value，错误解释也能自证。把 public term 和 operation 绑定后，T12 两次 16/16，T19 protection 18/18。这里证明的是 runtime evidence 能定位问题、component 机制能解决问题；最终 binding 还不是 Evolver 自主发现的。

第四步，我们验证 Evolver 会不会真的使用上一轮失败，而不是每轮重新猜。 T18 第一轮提出 warmup-boundary skill，Worker 也确实加载 skill、跑了 exactly-120 fixture，但 score 仍为 16/18。下一轮 Evolver 看到这个结果后，没有重复相同修改，而是明确排除它；剩余 percentile 与 NaN conventions 又无法从 public evidence 区分，于是选择 ABSTAIN。T24 则是另一个例子：第一版 completion guard 放在 `after_model`，但异常在这个 hook 之前就抛出；下一轮把它改成可达的 `wrap_model_call` middleware，真实 Worker 连续两次 empty response 后仍恢复并交付 artifact。只是组合后的 quant score 从 prior 16/17 降到 11/17，所以我们只说 delivery recovery 有效，不说整体 harness 更强。

第五步，我们把相同 experience interface 放到 QFBench 与 QuantCodeEval。 8 月 15 日完成 zero-model adapter preflight；8 月 16 日 live run 中，QFBench `swap-curve-bootstrap-ois` 的 task-only 与 history arms 都认为 public evidence 无法区分两个剩余解释，因此都没有修改。QuantCodeEval T26 则有更具体的 runtime contrast，Evolver 自主生成五文件 public-clause audit/revision component。两个 Worker 都调用了 component，但分别得到 14/17 和 12/17，说明“组件能激活”已经成立，“组件稳定涨分”仍未成立。T19 protection 为 18/18。整组 live experiment 使用 183 requests、8,231,008 tokens、USD 0.2101171912。

第六步，我们把 optimize task 的详细错误反馈给 Evolver，但没有给 Worker。 8 月 17 日，Evolver 能看到 T26 过去 13/17、14/17、12/17 的 item-level diagnostic，然后新增 quant-contract auditor 和 revise/re-audit workflow。Worker 只看到 public task、data 和 candidate harness。两次有效 Worker 都得到 16/17，全部 Type B properties 通过，Evolver 预测的 B5/B9 都转为 PASS。这个结果说明 rich evidence 确实帮助 Evolver 把错误定位到了更合适的 component，而不是把答案直接塞给 Worker。

第七步，我们把剩余 A10 从“分数差一点”拆成具体 estimator state。 Grid-only 仍然是 16/17。后续 causal ablation 发现，真正的问题是 CV complement 的非连续月份集合被下游重新变成首尾日期之间的连续区间，使 held-out fold 又进入 moment estimation；同时 fold-local scaling 复用了 full-sample state。修复这些 state semantics 后，trusted zero-model replay 达到 17/17。它证明

root cause 是对的，但因为不是 fresh Evolver-to-Worker lineage，所以当时还不能算自主 binary gain。

8 月 18 日的另一个负结果也很重要：R5 static component 能通过 synthetic wrong/correct contrasts，却让 fresh Worker 因 multi-output OLS shape mismatch 只得到 3/17，十一项 property crash。这说明 component 自测不能只检查源码规则，还需要执行 public entry point，检查 shape、finiteness 和 no-crash。

最后，我们验证 Evolver-produced harness 能不能在 fresh Worker 中真正拿到 binary reward。AP-1 给 parent harness 和 R3 candidate harness 同一个 3/17 artifact、同一个短程 repair budget。Parent 修到 12/17，candidate 修到 14/17，说明 candidate bundle 对 repair 有 +2 property 的帮助，但更慢、更贵。随后不再提供 seed artifact，而是让同一 candidate harness 从 public T26 task 开始工作。该 Worker 使用 59 requests、3,284,491 tokens、USD 0.120627420，最终达到 Type A 7/7、Type B 10/10、总计 17/17、reward 1。

1.3 七天结果速览

日期	主要实验	结果	这条结果现在怎么用
08-11	QFBench ME1-ME10	ME7、ME10 合法 ABSTAIN；无 candidate	证明终局控制可用，不是性能结果。
08-12	QuantCodeEval 五轮、v2、r8/r9	T24 最好 16/17；full-harness component 可生成和激活；r9 负向	从 prompt-only 转向 executable component，并保留历史。
08-13	T12 runtime invariant	初版 8/16；修复后两次 16/16；T19 18/18	证明 public semantic binding 有效，但当时仍有人工 seed。
08-14	T18 refine、T24 delivery	T18 16/18 后下一轮 ABSTAIN；T24 middleware 恢复 delivery，但组合为 11/17	证明负反馈、hook reachability 与 component interaction 可被区分。
08-15	Cross-benchmark preflight	QFBench/QuantCodeEval common adapter 可运行；无 model cost	只证明执行与 evidence surface，不证明涨分。
08-16	Cross-benchmark live breadth	QFBench 两个 arms 均 ABSTAIN；T26 13/17 → 14/17、repeat 12/17；T19 18/18	证明跨 benchmark 导航与组件激活，收益不稳定。
08-17	T26 answer-rich refine、causal ablation	两次 16/17；trusted repair 17/17	证明 rich evidence 与 estimator root cause；fresh autonomous binary 当时未闭合。
08-18	Estimator negative、AP-1、fresh candidate	15/17、3/17 negative；AP-1 12/17 vs 14/17；fresh 17/17	证明 bundle 有帮助并产生一次 fresh binary success；完整自主实验选择仍未验证。

1.4 案例研究 / Case Study

正向案例：从 repair gain 到 fresh 17/17。 同一个 3/17 artifact 在 parent harness 下修到 12/17，在 R3 candidate harness 下修到 14/17。随后相同 candidate harness 不再使用这个 seed artifact，而是面对 public T26 task 从头生成 strategy，最终在 official verifier 中通过全部 17 个 properties。这说明 harness 的作用不只是在旧答案上补丁式修复，它也能改变 fresh Worker 的完整工作过程。它还不是完整 autonomous search，因为 probe、promotion 和 final evaluation 是实验者安排的。

负向案例：component smoke 通过，但 Worker integration 失败。 R5 component 能区分 synthetic first/second-moment 与 public-scope wrong/correct pairs，也接受 retained 17/17 repair；fresh Worker 却写出了 shape 不兼容的 multi-output OLS helper，最后只有 3/17。这个结果告诉我们：下一步需要让 Evolver 自己跑一个短的 executable probe，拿到真实 traceback，再决定是否保留或修改 component，而不是继续把 static rule list 写得 longer。

2. 分析 / Analysis

2.1 我们现在已经验证了什么

第一，**Evolver 已经不只是能改 prompt。** 它可以新增和修改 tool、skill、middleware、agent binding 与 system prompt，真实 Worker 也确实调用过这些组件。

第二，**Evolver 能看到并使用历史。** 它能读取上一轮 candidate、diff、artifact、trace 和 score；T18 的第二轮确实用第一轮的无收益结果排除了原 hypothesis，T24 也根据 hook 不可达的结果换了 middleware locus。因此我们缺的不是“历史根本看不到”，而是让 Evolver 自主选择下一条最有信息量的实验。

第三，**rich evidence 在 T26 上确实有帮助。** 在只给 aggregate answer-free feedback 时，T26 component 的结果是 14/17、12/17，方向不稳定。给 Evolver item-level optimize diagnostic、同时保持 Worker blind 后，两个独立 Worker 都达到 16/17，B5/B9 都按预测通过。这是目前 rich-evidence mechanism 最直接的证据。

第四，**Evolver-produced harness 已经有一次 fresh binary result。** T26 fresh Worker 的 17/17 说明这条路线不只是能改善小分，至少在一个 task 上已经跨过 binary gate。

2.2 现在还没有验证什么

我们还没有跑过这样一条完整路径：从 shell-only H0 开始，Evolver 自己看 H0 artifact 和 trace，自己决定改哪个 component，自己决定跑什么 Worker probe，看到 probe 结果后再做第二次决定，最后由 fresh Worker 独立评分。

当前 17/17 只完成了其中后半段。R3 candidate 是 Evolver 产生的，fresh Worker 和 official verifier 也是真实的；但 AP-1 的 seed artifact、repair instruction、晋级规则和 final confirmation 是实验者安排的。因此，当前可以说“candidate harness 能 work”，还不能说“整套 search 已经完全自主”。

我们也没有验证多 task 稳定提升。T19 是 protection，不是与 T26 同 failure mechanism 的 positive transfer；QFBench 当前只验证了 common interface 与 calibrated ABSTAIN，没有 score

improvement。全量 QuantCodeEval、完整 QFBench panel、held-out test 和 scheduler 都还没有进入当前结论。

2.3 为什么下一步不是继续加 failure class

T26 已经给了两个很清楚的教训。第一，grid resolution 看起来像 A10 的原因，但单独增加 grid 并不能通过 A10。第二，R5 static checks 看起来覆盖了更多 estimator semantics，Worker 却因为普通 array-shape integration bug 只得 3/17。

所以 domain specialization 仍然有用，但应该表现为 Evolver 可以检查的 state，例如 sample membership、moment scope、unit state、temporal endpoint、runtime shape；它不能变成一份越来越长的答案枚举。更重要的是，Evolver 要能自己选择一个 executable observation 来区分这些解释。

3. 当前问题与结论边界 / Problems and Open Questions

1. **自主 experiment choice 未验证。** 这是 AP-2M 要解决的问题，也是当前离“完整自主搜索”最近的 gap。
2. **H0 bootstrap 未验证。** AP-3 才回答系统能否不依赖历史高分 candidate，从 shell-only H0 起步。
3. **17/17 尚未重复。** 现在能说明 feasibility，不能说明稳定性；先验证自主闭环，再决定是否重复当前 candidate 或重复整个 AP-3 lineage。
4. **Component contribution 还没有完全拆开。** AP-1 的 candidate bundle 有帮助，但 static auditor 对 14/17 artifact 仍然误报，收益可能来自 contract reading、revision workflow、runtime checks 及其组合。
5. **Cross-task performance 未验证。** T26 是一个 task；T19 只是 protection；QFBench 目前没有 performance gain。
6. **Scheduler 尚未实现。** 当前没有渐进式、异步、成本感知的多 task evaluation。这个缺口真实存在，但它属于 scale-up，不是 AP-2M 前置条件。

4. 下一步实验计划 / Next Experiment Plan

P1 — 先把 AP-2M 的最小闭环接起来

在现有 QuantCodeEval runner 上增加两个 Evolver rounds 和一个 `experiment_spec`：

- Round 1 自己选择 history、artifact 或 from-scratch mode；
- 自己写 Worker instruction、选择 component 和 10–12 iteration probe budget；
- 写清 prediction，以及看到什么结果会改变当前判断；
- coordinator 只执行这一条合法 probe；

- Round 2 收到 artifact、trace、official optimize properties、runtime、requests、tokens 和 cost；
- Round 2 再选择 retain、refine、rollback、compose、submit 或 ABSTAIN；
- final candidate 用一个独立 T26 Worker 和 unchanged official verifier 评分。

实现上只复用现有 runner、Worker executor、candidate workspace 和 verifier，结果写普通 JSON。暂时不建设通用 experience service、复杂 durable notebook 或 scheduler。

P2 — 用一次真实 AP-2M canary 验证自主实验能力

使用 pre-AP-1 history，两轮 Evolver、一个短 Worker probe、一个合法 candidate 的 final Worker，concurrency 1，建议 provider cap USD 0.25。不给 AP-1 人工 repair prompt，也不给 expert root-cause summary。

这一轮主要看四件事：

1. Evolver 是否真的自己选择了 probe；
2. Round 2 是否因为 Round 1 结果改变了决定；
3. 自己选择的 component 是否修复 predicted fault、提高 properties 或减少成本；
4. final candidate 是否达到或保持 17/17。

AP-2M 不需要“超过 17/17”。它的核心目标是把目前分开的两条证据连起来：一条是 Evolver 会做 component search，另一条是 candidate harness 能拿到 17/17。

P3 — AP-2M 成立后，再跑 AP-3 H0 bootstrap

AP-3 只提供 shell-only H0、public T26 task，以及 AP-3 内部 fresh H0 Worker 产生的 artifact、trace、score、runtime 和 optimize diagnostic。不给 R3/AP-1 candidate、历史高分 artifact、人工 repair prompt 或已知 root cause。

仍然使用两轮 Evolver 和一个 intermediate probe。Final score 高于该次 fresh H0，就算初步 bootstrap helpful；达到 17/17 才算 binary helpful。

P4 — 最后才做 repeat、第二个 task 和 scheduler

如果 AP-3 positive，再按顺序做：

1. fresh Evolver repeat；
2. 一个 failure mechanism 不同的 QuantCodeEval task；
3. 一个 QFBench canary；
4. 渐进式、异步、成本感知 scheduler；
5. 冻结 candidate 后运行较大 test set。